Lab Assignment 4 — NLP Preprocessing and Text Classification

Course: DEEP LEARNING MDM
Assignment Title: NLP Preprocessing and Text Classification
Assignment Title: Nilesh Raju Sarule
PRN: 202301070012
Institution: MIT Academy of Engineering (MITAOE), Alandi, Pune

---

Objective

Implement NLP preprocessing techniques and build a text classification model using machine learning.

Learning Outcomes

- Apply NLP preprocessing: tokenization, stopword removal, stemming, lemmatization
- Implement text vectorization: TF-IDF and CountVectorizer (BoW)
- Build a machine learning classification model (Naive Bayes + Logistic Regression)
- Evaluate model performance using accuracy, precision, recall, F1-score

---

Table of Contents

1. Setup & Installations
2. Dataset Preparation
3. NLP Preprocessing Pipeline
    - 3.1 Tokenization
    - 3.2 Stop Word Removal
    - 3.3 Stemming
    - 3.4 Lemmatization
4. Text Vectorization
    - 4.1 Bag of Words (CountVectorizer)
    - 4.2 TF-IDF Vectorizer
5. Text Classification Models
    - 5.1 Naive Bayes Classifier
    - 5.2 Logistic Regression Classifier
6. Model Evaluation & Comparison
7. Visualization
8. Prediction on New Samples
9. Summary & Conclusions

---

1. Setup & Installations

Install and import all required libraries for this assignment.

```
# Install required packages
!pip install nltk scikit-learn matplotlib seaborn pandas --quiet
print(" Packages installed successfully!")
```

```
 Packages installed successfully!
```

```
import nltk

# Download all required NLTK resources
resources = [
    ('tokenizers/punkt',              'punkt'),
    ('tokenizers/punkt_tab',          'punkt_tab'),
    ('corpora/stopwords',             'stopwords'),
    ('taggers/averaged_perceptron_tagger_eng', 'averaged_perceptron_tagger_eng'),
    ('corpora/wordnet',               'wordnet'),
    ('sentiment/vader_lexicon',       'vader_lexicon'),
]

for path, name in resources:
```

```
    try:
        nltk.data.find(path)
        print(f"  ✓ '{name}' already available")
    except LookupError:
        nltk.download(name, quiet=True)
        print(f"  ⬇ '{name}' downloaded")

print("\n All NLTK resources ready!")
```

```
  ✓ 'punkt' already available
  ✓ 'punkt_tab' already available
  ✓ 'stopwords' already available
  ✓ 'averaged_perceptron_tagger_eng' already available
  ⬇ 'wordnet' downloaded
  ⬇ 'vader_lexicon' downloaded

 All NLTK resources ready!
```

```
# Core imports
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# NLTK
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Scikit-learn
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)

# Settings
plt.style.use('seaborn-v0_8-whitegrid')
RANDOM_STATE = 42

print(" All imports successful!")
```

```
 All imports successful!
```

---

2. Dataset Preparation

We use a **multi-class news headline dataset** covering 4 categories:

- **Technology** — AI, software, gadgets
- **Sports** — Cricket, football, Olympics
- **Politics** — Elections, policy, government
- **Health** — Medicine, fitness, research

We use **50 headlines per class (200 total)** to ensure sufficient training data for reliable model evaluation.

```
# ----------------------------------------------------------------------
# Dataset: 200 labelled news headlines (50 per class)
# Expanded from 40 to 200 samples for better model training & evaluation
# ----------------------------------------------------------------------
data = {
    'text': [
        # --- Technology (label: 0) --- 50 headlines
        "OpenAI releases GPT-5 with improved reasoning capabilities",
        "Apple unveils new MacBook Pro with M3 Ultra chip",
        "Google launches Gemini AI model to compete with ChatGPT",
        "Microsoft integrates Copilot AI into all Office applications",
        "Tesla self-driving software receives major update this quarter",
        "Meta announces open-source large language model for developers",
        "Quantum computing breakthrough achieved by IBM researchers",
        "New cybersecurity vulnerability discovered in popular software",
        "Samsung unveils foldable smartphone with improved display technology",
        "Cloud computing market grows rapidly as businesses shift to AWS",
        "SpaceX successfully launches Starship rocket into orbit",
        "NVIDIA releases new GPU with groundbreaking AI performance",
        "Amazon Web Services expands data center infrastructure globally",
```

```
"Apple Vision Pro mixed reality headset ships to customers worldwide",
"Researchers develop new battery technology with longer lifespan",
"Twitter rebrands to X under Elon Musk leadership",
"Google DeepMind creates AI system that solves complex math problems",
"Intel releases next generation processor with enhanced performance",
"Microsoft Azure cloud platform reports record revenue growth",
"Robotics startup unveils humanoid robot for industrial use",
"New programming language gains popularity among developers worldwide",
"Autonomous vehicle startup raises funding for fleet expansion",
"Cybersecurity firm detects massive data breach affecting millions",
"5G network rollout accelerates across major metropolitan areas",
"Blockchain technology adopted by major financial institutions globally",
"Machine learning algorithm outperforms doctors in cancer detection",
"Social media platform launches new privacy features for users",
"Tech giant acquires startup specializing in natural language processing",
"Semiconductor shortage continues to impact global electronics supply",
"Virtual reality gaming headset sells out within hours of launch",
"Open source community releases major update to popular framework",
"Artificial intelligence startup valued at ten billion dollars",
"New encryption standard adopted by government agencies worldwide",
"Electric vehicle charging network expands to rural areas",
"Drone delivery service approved for commercial operations in cities",
"Tech company launches satellite internet service for remote regions",
"Scientists achieve breakthrough in photonic computing technology",
"Software vulnerability patched after millions of devices affected",
"Wearable health monitor receives FDA approval for consumer market",
"AI-generated art wins major competition sparking copyright debate",
"New operating system update improves battery life on laptops",
"Tech startup develops carbon capture technology using machine learning",
"Gaming company announces next generation console release date",
"Streaming service adds spatial audio feature for immersive experience",
"Autonomous robot performs surgery with precision surpassing human doctors",
"Browser company releases privacy-focused search engine globally",
"Mobile payment adoption surges in developing countries worldwide",
"Chip manufacturer announces new fabrication process at three nanometers",
"Edge computing reduces latency for real-time applications significantly",
"Digital twin technology transforms manufacturing and logistics industries",

# --- Sports (label: 1) --- 50 headlines
"India wins cricket World Cup final against Australia in thrilling match",
"Virat Kohli scores century in test match against England",
"Real Madrid wins UEFA Champions League title this season",
"Lionel Messi announces retirement from professional football",
"Olympic gold medal won by Indian athlete in javelin throw",
"IPL auction sets record as players sold for highest bids",
"Roger Federer inducted into Tennis Hall of Fame ceremony",
"NBA Finals won by Boston Celtics defeating Dallas Mavericks",
"Formula 1 driver breaks lap record at Monaco Grand Prix",
"National football team qualifies for FIFA World Cup 2026",
"Neeraj Chopra wins gold medal at World Athletics Championship",
"Manchester City wins Premier League title for consecutive season",
"Rafael Nadal announces retirement from professional tennis career",
"LeBron James becomes all-time leading scorer in NBA history",
"India beats Pakistan in high-scoring cricket ODI match",
"Swimmer breaks world record in Olympic finals at Paris games",
"Wrestler wins gold at Commonwealth Games in historic performance",
"Cricket board announces new rules for international test matches",
"Marathon runner sets new world record in Berlin race",
"Badminton player wins All England Open championship title",
"Football club signs record-breaking transfer for new striker",
"Olympic torch relay begins its journey across host country",
"Tennis grand slam champion defeats rival in five-set thriller",
"Hockey team wins bronze medal at Asian Games tournament",
"Basketball star returns from injury to lead team to victory",
"Golf player wins Masters tournament in dramatic playoff finish",
"Boxing champion retains title in unanimous decision victory",
"Cycling race won by mountain climber in stunning Alpine stage",
"Volleyball team reaches semifinal at World Championship tournament",
"Archery team wins gold at international championship competition",
"Rugby world cup final draws record television viewership globally",
"Sprinter clocks fastest time of season at Diamond League meet",
"Football manager resigns after poor run of results this season",
"Young cricketer makes debut with stunning century against rivals",
"Swimming federation announces new format for championship events",
"Sports federation bans athlete for doping violation worldwide",
"Cricket stadium breaks attendance record for test match day",
"Weightlifter sets new personal best at Asian championships",
"Tennis player overcomes injury to win tournament title match",
"Football player wins prestigious award for outstanding season performance",
"Winter Olympics host city announces venue construction completion",
"Gymnastics champion wins multiple gold medals at world event",
"Table tennis star dominates tournament winning all matches played",
```

```
"Kabaddi team wins gold medal at South Asian Games",
"Stadium upgrade project approved for upcoming international tournament",
"Para-athlete wins gold at World Para Athletics Championships",
"Youth cricket team wins Under-19 World Cup championship title",
"Sports ministry announces new funding scheme for athlete development",
"Coach leads national team to historic victory after decade",
"Triathlon champion completes race in record-breaking finishing time",

# --- Politics (label: 2) --- 50 headlines
"Parliament passes new budget with increased education funding",
"Prime Minister announces new economic policy to boost growth",
"General elections scheduled for next month across all states",
"United Nations holds emergency meeting on climate agreement",
"Government introduces digital ID scheme for all citizens",
"Opposition party wins state election in surprise landslide victory",
"New foreign policy framework signed between two nations",
"President signs executive order on immigration reform",
"Senate votes to approve new infrastructure spending bill",
"Supreme Court delivers landmark ruling on data privacy rights",
"Diplomatic talks resume between neighboring countries after tensions",
"Government unveils five-year national development plan with targets",
"International court issues arrest warrant for former head of state",
"Treaty negotiations conclude with historic multilateral agreement signed",
"Minister resigns amid corruption allegations and public controversy",
"Election commission announces voter registration deadline for citizens",
"G20 summit focuses on global economic recovery and debt relief",
"Constitutional amendment approved after referendum with strong majority",
"Trade deal signed between countries reducing tariffs on goods",
"Protest erupts in capital over new taxation policy proposal",
"Government launches anti-corruption drive targeting public officials",
"Parliament debates controversial bill on media regulation and freedom",
"Foreign minister visits ally nation to strengthen bilateral relations",
"New cabinet formation announced following coalition negotiations",
"Regional summit addresses cross-border environmental pollution issues",
"Government announces free vaccination program for all age groups",
"Opposition leader faces legal challenge over election campaign finances",
"City mayor announces ambitious urban development and housing plan",
"UN Security Council discusses peacekeeping mission mandate renewal",
"Government approves landmark social welfare scheme for rural poor",
"Prime minister delivers address on national unity and progress",
"Political party holds national convention to elect new leadership",
"Defense minister announces military modernization and procurement plan",
"Parliament approves new education policy reforming school curriculum",
"Voter turnout reaches record high in recent state elections",
"Government implements new minimum wage policy for all workers",
"Leaders agree on carbon neutrality targets at climate summit",
"New law passed protecting freedom of press and journalists",
"Government launches rural electrification scheme in remote areas",
"Prime minister meets business leaders to discuss economic reforms",
"International sanctions imposed on country over human rights violations",
"Government announces major infrastructure project connecting cities",
"Political scandal emerges days before crucial national election vote",
"Parliamentary session opens with debate on national budget priorities",
"New ambassador appointed to strengthen diplomatic ties with partner",
"Government rolls out national broadband connectivity scheme today",
"Opposition coalition forms strategic alliance ahead of general elections",
"Court strikes down controversial election law as unconstitutional",
"Governor declares state of emergency after natural disaster strikes",
"International observers praise election conduct as free and fair",

# --- Health (label: 3) --- 50 headlines
"New vaccine shows 95 percent effectiveness against disease in trial",
"WHO announces global health emergency due to new virus outbreak",
"Study finds Mediterranean diet reduces heart disease risk significantly",
"AI-powered diagnostic tool detects cancer earlier than traditional methods",
"Daily exercise shown to improve mental health outcomes in research",
"Breakthrough drug approved for treatment of Alzheimer disease",
"Sleep deprivation linked to increased obesity risk in latest study",
"Gene therapy trial shows promise for rare genetic disorder patients",
"Yoga and meditation reduce stress hormone levels says new research",
"Hospital adopts robotic surgery system improving patient recovery times",
"New antibiotic effective against drug-resistant bacteria in clinical trial",
"Mental health awareness campaign launches nationwide across all states",
"Researchers discover genetic marker linked to increased cancer risk",
"Blood pressure medication reduces stroke risk in elderly patients",
"Pandemic preparedness plan updated following lessons from COVID-19",
"Telemedicine adoption increases access to healthcare in rural communities",
"New diagnostic test detects multiple cancers from single blood sample",
"Childhood obesity rates decline following school nutrition program",
"Heart transplant patient achieves milestone of living ten years post-surgery",
"Research links air pollution to cognitive decline in aging populations",
"Universal health coverage expanded to include dental and vision care",
```

```
        "New mental health app shows positive outcomes in clinical validation study",
        "Scientists map complete human immune system in landmark research",
        "Drug trial shows breakthrough results for treatment-resistant depression",
        "Hospitals face nursing shortage impacting quality of patient care",
        "Stem cell therapy restores vision in patients with degenerative disease",
        "Nutrition guidelines updated to reflect new research on healthy eating",
        "Infectious disease outbreak contained after swift public health response",
        "Researchers develop wearable device to monitor chronic disease remotely",
        "Health ministry launches nationwide immunization drive for children",
        "Study shows regular walking reduces risk of premature death significantly",
        "New surgical technique reduces complication rates in complex operations",
        "Diabetes management improves with new continuous glucose monitoring device",
        "Researchers identify protein responsible for Parkinson disease progression",
        "Childhood vaccination rates drop raising concerns about disease outbreaks",
        "Hospital launches program to reduce antibiotic overuse and resistance",
        "Study finds link between gut microbiome and mental health disorders",
        "New pain management protocol reduces opioid dependency in patients",
        "Cancer survival rates improve significantly due to early detection programs",
        "Research shows mindfulness practice reduces anxiety and blood pressure",
        "Organ donation drive increases availability for transplant patients",
        "Health insurance reforms proposed to expand coverage for low-income families",
        "New test diagnoses Alzheimer disease ten years before symptom onset",
        "Fitness program for elderly reduces fall risk and improves mobility",
        "Researchers find link between vitamin D deficiency and autoimmune disorders",
        "Hospital emergency departments report surge in respiratory illness cases",
        "Study shows plant-based diet reduces risk of cardiovascular disease",
        "New therapy shows promise for treating treatment-resistant epilepsy patients",
        "Global health initiative aims to eliminate malaria by twenty thirty-five",
        "Research reveals impact of social isolation on long-term health outcomes",
    ],
    'label': [0]*50 + [1]*50 + [2]*50 + [3]*50
}

label_map = {0: 'Technology', 1: 'Sports', 2: 'Politics', 3: 'Health'}

df = pd.DataFrame(data)
df['category'] = df['label'].map(label_map)

print(f" Dataset shape: {df.shape}")
print(f"\n Class distribution:")
print(df['category'].value_counts())
print("\n Sample rows:")
df[['text','category']].head(8)
```

```
 Dataset shape: (200, 3)

 Class distribution:
category
Technology    50
Sports        50
Politics      50
Health        50
Name: count, dtype: int64

 Sample rows:
```

	text	category
0	OpenAI releases GPT-5 with improved reasoning ...	Technology
1	Apple unveils new MacBook Pro with M3 Ultra chip	Technology
2	Google launches Gemini AI model to compete wit...	Technology
3	Microsoft integrates Copilot AI into all Offic...	Technology
4	Tesla self-driving software receives major upd...	Technology
5	Meta announces open-source large language mode...	Technology
6	Quantum computing breakthrough achieved by IBM...	Technology
7	New cybersecurity vulnerability discovered in ...	Technology

3. NLP Preprocessing Pipeline

Each raw text goes through a 5-step cleaning pipeline before modelling:

```
Raw Text → Lowercase → Remove Noise → Tokenize → Remove Stop Words → Stem/Lemmatize
```

3.1 Tokenization

What: Splits raw text into individual word tokens.
Why: Models process sequences of tokens, not raw strings.

```
sample = df['text'][0]
print(f" Original text:\n  '{sample}'\n")

# Word tokenization
word_tokens = word_tokenize(sample)
print(f" Word Tokens ({len(word_tokens)} tokens):")
print(f"  {word_tokens}")

# Sentence tokenization demo
multi = "AI is transforming industries. Healthcare is the biggest beneficiary. Education is next."
sent_tokens = sent_tokenize(multi)
print(f"\n Sentence Tokens from multi-sentence string:")
for i, s in enumerate(sent_tokens, 1):
    print(f"  Sentence {i}: {s}")
```

```
 Original text:
  'OpenAI releases GPT-5 with improved reasoning capabilities'

 Word Tokens (7 tokens):
  ['OpenAI', 'releases', 'GPT-5', 'with', 'improved', 'reasoning', 'capabilities']

 Sentence Tokens from multi-sentence string:
  Sentence 1: AI is transforming industries.
  Sentence 2: Healthcare is the biggest beneficiary.
  Sentence 3: Education is next.
```

3.2 Stop Word Removal

What: Removes high-frequency words ("is", "the", "a") that carry little meaning.
Why: Reduces noise and dimensionality, keeps only content-bearing words.

```
stop_words = set(stopwords.words('english'))
print(f" NLTK English stop words count: {len(stop_words)}")
print(f" Sample stop words: {sorted(list(stop_words))[:15]}\n")

# Apply to our sample
tokens_lower = [t.lower() for t in word_tokens]
filtered = [t for t in tokens_lower if t.isalpha() and t not in stop_words]

print(f" Before removal ({len(tokens_lower)} tokens): {tokens_lower}")
print(f" After removal  ({len(filtered)} tokens): {filtered}")
print(f"\n Tokens removed: {len(tokens_lower) - len(filtered)} ({(1 - len(filtered)/len(tokens_lower))*100:.1f}% reduction)
```

```
 NLTK English stop words count: 198
 Sample stop words: ['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'are

 Before removal (7 tokens): ['openai', 'releases', 'gpt-5', 'with', 'improved', 'reasoning', 'capabilities']
 After removal  (5 tokens): ['openai', 'releases', 'improved', 'reasoning', 'capabilities']

 Tokens removed: 2 (28.6% reduction)
```

3.3 Stemming

What: Strips word suffixes to a common root using rule-based heuristics (PorterStemmer).
Why: Groups word variants — faster than lemmatization but may produce non-real words.

```
stemmer = PorterStemmer()

test_words = ['running', 'released', 'improving', 'technologies',
              'computational', 'reasoning', 'capabilities', 'studies']

print("  Stemming Results (PorterStemmer):")
print(f"  {'Original':<20} → {'Stemmed'}")
print("  " + "-"*35)
for word in test_words:
    print(f"  {word:<20} → {stemmer.stem(word)}")

# Stem the filtered tokens from our sample
stemmed_sample = [stemmer.stem(t) for t in filtered]
print(f"\n Sample headline after stemming:")
print(f"  {stemmed_sample}")
```

```
  Stemming Results (PorterStemmer):
  Original             → Stemmed
  ----------------------------------
  running              → run
  released             → releas
  improving            → improv
  technologies         → technolog
  computational        → comput
  reasoning            → reason
  capabilities         → capabl
  studies              → studi

 Sample headline after stemming:
  ['openai', 'releas', 'improv', 'reason', 'capabl']
```

3.4 Lemmatization

What: Maps each word to its true base form using a dictionary (WordNet).
Why: Produces real words with correct meaning — preferred for text classification.

```
lemmatizer = WordNetLemmatizer()

# Word-POS pairs for demonstration
pairs = [
    ('running',      'v'),
    ('released',     'v'),
    ('capabilities', 'n'),
    ('technologies', 'n'),
    ('better',       'a'),
    ('studies',      'v'),
    ('geese',        'n'),
    ('was',          'v'),
]

print(" Lemmatization Results (WordNetLemmatizer):")
print(f"  {'Original':<20} {'POS':<6} → {'Lemma'}")
print("  " + "-"*40)
for word, pos in pairs:
    lemma = lemmatizer.lemmatize(word, pos=pos)
    print(f"  {word:<20} {pos:<6} → {lemma}")

# Lemmatize filtered sample tokens (use 'v' as default POS)
lemmatized_sample = [lemmatizer.lemmatize(t, pos='v') for t in filtered]
print(f"\n Sample headline after lemmatization:")
print(f"  {lemmatized_sample}")
```

```
 Lemmatization Results (WordNetLemmatizer):
  Original             POS    → Lemma
  ----------------------------------------
  running              v      → run
  released             v      → release
  capabilities         n      → capability
  technologies         n      → technology
  better               a      → good
  studies              v      → study
  geese                n      → goose
  was                  v      → be

 Sample headline after lemmatization:
  ['openai', 'release', 'improve', 'reason', 'capabilities']
```

3.5 Full Preprocessing Function

Combine all steps into a single reusable function applied to every text in the dataset.

```
def preprocess_text(text, use_stemming=False):
    """
    Full NLP preprocessing pipeline:
      1. Lowercase
      2. Remove non-alphabetic characters
      3. Tokenize
      4. Remove stop words
      5. Stem or Lemmatize
    """
    # Step 1: Lowercase
    text = text.lower()

    # Step 2: Remove punctuation, numbers, special chars
    text = re.sub(r'[^a-z\s]', '', text)
```

```
    # Step 3: Tokenize
    tokens = word_tokenize(text)

    # Step 4: Remove stop words and single characters
    tokens = [t for t in tokens if t not in stop_words and len(t) > 1]

    # Step 5: Normalize
    if use_stemming:
        tokens = [stemmer.stem(t) for t in tokens]
    else:
        tokens = [lemmatizer.lemmatize(t, pos='v') for t in tokens]

    return ' '.join(tokens)


# Apply to full dataset
df['processed_text'] = df['text'].apply(preprocess_text)

print(" Preprocessing applied to all samples.\n")
print(" Before vs After comparison:")
print("-" * 75)
for i in [0, 50, 100, 150]:
    print(f"[{df['category'][i]}]")
    print(f"  RAW  : {df['text'][i]}")
    print(f"  CLEAN: {df['processed_text'][i]}")
    print()
```

```
 Preprocessing applied to all samples.

 Before vs After comparison:
---------------------------------------------------------------------------
[Technology]
  RAW  : OpenAI releases GPT-5 with improved reasoning capabilities
  CLEAN: openai release gpt improve reason capabilities

[Sports]
  RAW  : India wins cricket World Cup final against Australia in thrilling match
  CLEAN: india win cricket world cup final australia thrill match

[Politics]
  RAW  : Parliament passes new budget with increased education funding
  CLEAN: parliament pass new budget increase education fund

[Health]
  RAW  : New vaccine shows 95 percent effectiveness against disease in trial
  CLEAN: new vaccine show percent effectiveness disease trial
```

---

4. Text Vectorization

Machine learning models require **numerical** input. We convert text to vectors using two methods.

Method	Description	Weakness
BoW (CountVectorizer)	Word frequency counts	Ignores importance
TF-IDF	Weighted frequency	Still ignores order

4.1 Bag of Words (CountVectorizer)

```
# Demonstrate BoW on a mini corpus
mini_corpus = [
    "AI model learns language",
    "language model uses deep learning",
    "cricket match India wins",
    "India cricket team wins match"
]

bow_demo = CountVectorizer()
X_demo = bow_demo.fit_transform(mini_corpus)
vocab = bow_demo.get_feature_names_out()

df_bow = pd.DataFrame(
    X_demo.toarray(),
    columns=vocab,
    index=[f"Doc{i+1}" for i in range(len(mini_corpus))]
)
print(" Bag of Words Matrix (mini demo):")
print(df_bow)
print(f"\n Vocabulary size: {len(vocab)} unique words")
print(f" Matrix shape: {X_demo.toarray().shape} (docs × vocab)")
```

```
 Bag of Words Matrix (mini demo):
      ai  cricket  deep  india  language  learning  learns  match  model  \
Doc1   1        0     0      0         1         0       1      0      1
Doc2   0        0     1      0         1         1       0      0      1
Doc3   0        1     0      1         0         0       0      1      0
Doc4   0        1     0      1         0         0       0      1      0

      team  uses  wins
Doc1     0     0     0
Doc2     0     1     0
Doc3     0     0     1
Doc4     1     0     1

 Vocabulary size: 12 unique words
 Matrix shape: (4, 12) (docs × vocab)
```

4.2 TF-IDF Vectorizer

TF-IDF Formula:

$$\text{TF-IDF}(t, d) = TF(t, d) \times \log\left(\frac{N}{1 + df(t)}\right)$$

- **TF** = how often term `t` appears in document `d`
- **IDF** = penalises terms that appear in many documents

```
# TF-IDF on full dataset — build vocabulary from processed text
tfidf_vec = TfidfVectorizer(max_features=500, ngram_range=(1, 2))
X_tfidf = tfidf_vec.fit_transform(df['processed_text'])

print(f" TF-IDF matrix shape: {X_tfidf.shape}")
print(f" Vocabulary size (top-{500} unigrams+bigrams): {len(tfidf_vec.get_feature_names_out())}")

# Show top 5 keywords per category
print("\n Top-5 TF-IDF keywords per category:")
print("-" * 55)
for cat_label, cat_name in label_map.items():
    indices = df[df['label'] == cat_label].index
    cat_matrix = X_tfidf[indices]
    mean_scores = cat_matrix.mean(axis=0).A1
    top_idx = mean_scores.argsort()[::-1][:5]
    top_words = [tfidf_vec.get_feature_names_out()[i] for i in top_idx]
    print(f"  {cat_name:<12}: {top_words}")
```

```
 TF-IDF matrix shape: (200, 500)
 Vocabulary size (top-500 unigrams+bigrams): 500

 Top-5 TF-IDF keywords per category:
-------------------------------------------------------
  Technology  : ['technology', 'release', 'new', 'launch', 'startup']
  Sports      : ['win', 'world', 'team', 'tennis', 'match']
  Politics    : ['government', 'new', 'policy', 'minister', 'election']
  Health      : ['disease', 'health', 'reduce', 'show', 'new']
```

---

5. Text Classification Models

We train and compare two classifiers:

- **Multinomial Naive Bayes** — probabilistic baseline, great for text
- **Logistic Regression** — linear discriminative model, typically stronger

```
# ----------------------------------------------------------------------
# Prepare features and labels — use TF-IDF as the feature representation
# ----------------------------------------------------------------------
X = X_tfidf
y = df['label'].values

# 75% train / 25% test split — stratified to keep class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=RANDOM_STATE,
    stratify=y
)

print(f" Training samples : {X_train.shape[0]}")
print(f" Testing  samples : {X_test.shape[0]}")
print(f" Feature dimensions: {X_train.shape[1]}")
```

```
print(f"\n Train class distribution: {dict(zip(*np.unique(y_train, return_counts=True)))}")
print(f" Test  class distribution: {dict(zip(*np.unique(y_test,  return_counts=True)))}")
```

```
 Training samples : 150
 Testing  samples : 50
 Feature dimensions: 500

 Train class distribution: {np.int64(0): np.int64(37), np.int64(1): np.int64(38), np.int64(2): np.int64(37), np.int64(3): np
 Test  class distribution: {np.int64(0): np.int64(13), np.int64(1): np.int64(12), np.int64(2): np.int64(13), np.int64(3): np
```

5.1 Naive Bayes Classifier

Theory: Uses Bayes' theorem — assumes word occurrences are conditionally independent given the class.

$$P(\text{class}|\text{text}) \propto P(\text{class}) \cdot \prod_{w} P(w|\text{class})$$

```
# Train Naive Bayes
nb_model = MultinomialNB(alpha=1.0)   # alpha=1.0 → Laplace smoothing
nb_model.fit(X_train, y_train)

y_pred_nb = nb_model.predict(X_test)

print(" Naive Bayes — Classification Report:")
print("=" * 60)
print(classification_report(
    y_test, y_pred_nb,
    target_names=list(label_map.values()),
    zero_division=0
))
print(f" Overall Accuracy: {accuracy_score(y_test, y_pred_nb):.2%}")
```

```
 Naive Bayes — Classification Report:
============================================================
              precision    recall  f1-score   support

  Technology       0.85      0.85      0.85        13
      Sports       0.80      1.00      0.89        12
    Politics       1.00      0.77      0.87        13
      Health       0.92      0.92      0.92        12

    accuracy                           0.88        50
   macro avg       0.89      0.88      0.88        50
weighted avg       0.89      0.88      0.88        50

 Overall Accuracy: 88.00%
```

5.2 Logistic Regression Classifier

Theory: Models the probability of each class using a softmax function over a linear combination of features.

$$P(y = k|x) = \frac{e^{w_k \cdot x}}{\sum_j e^{w_j \cdot x}}$$

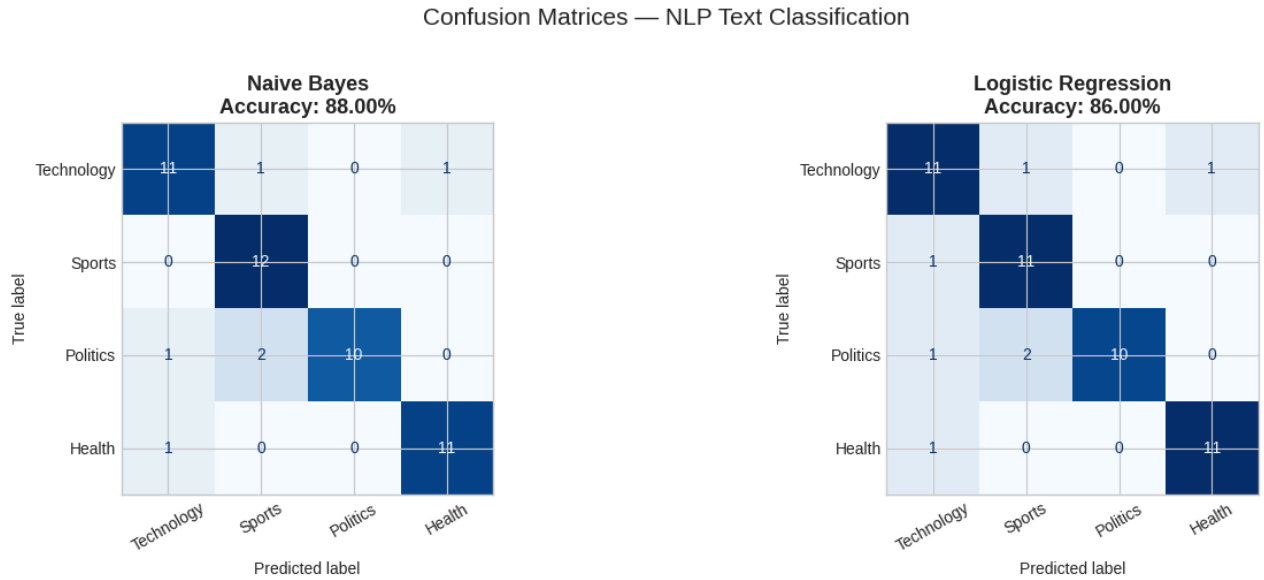
```
# Train Logistic Regression
# Note: multi_class='multinomial' is deprecated in newer sklearn versions.
# Using solver='lbfgs' with default multi_class='auto' which selects multinomial automatically.
lr_model = LogisticRegression(
    max_iter=1000,
    solver='lbfgs',
    random_state=RANDOM_STATE,
    C=1.0
)
lr_model.fit(X_train, y_train)

y_pred_lr = lr_model.predict(X_test)

print(" Logistic Regression — Classification Report:")
print("=" * 60)
print(classification_report(
    y_test, y_pred_lr,
    target_names=list(label_map.values()),
    zero_division=0
))
print(f" Overall Accuracy: {accuracy_score(y_test, y_pred_lr):.2%}")
```

```
 Logistic Regression — Classification Report:
============================================================
              precision    recall  f1-score   support
```

```
   Technology       0.79      0.85      0.81        13
       Sports       0.79      0.92      0.85        12
     Politics       1.00      0.77      0.87        13
       Health       0.92      0.92      0.92        12

     accuracy                           0.86        50
    macro avg       0.87      0.86      0.86        50
 weighted avg       0.87      0.86      0.86        50

 Overall Accuracy: 86.00%
```

6. Model Evaluation & Comparison

```
# ----------------------------------------------------------------------
# Side-by-side confusion matrices
# ----------------------------------------------------------------------
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
class_names = list(label_map.values())

for ax, y_pred, title in zip(
    axes,
    [y_pred_nb, y_pred_lr],
    ['Naive Bayes', 'Logistic Regression']
):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'{title}\nAccuracy: {accuracy_score(y_test, y_pred):.2%}',
                 fontsize=13, fontweight='bold')
    ax.tick_params(axis='x', rotation=30)

plt.suptitle('Confusion Matrices — NLP Text Classification', fontsize=15, y=1.02)
plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Confusion matrices saved.")
```




```
 Confusion matrices saved.
```

```
# ----------------------------------------------------------------------
# Metrics comparison table
# ----------------------------------------------------------------------
from sklearn.metrics import precision_score, recall_score, f1_score

def get_metrics(y_true, y_pred, model_name):
    return {
        'Model': model_name,
        'Accuracy':  round(accuracy_score(y_true, y_pred), 4),
        'Precision': round(precision_score(y_true, y_pred, average='weighted', zero_division=0), 4),
        'Recall':    round(recall_score(y_true, y_pred, average='weighted', zero_division=0), 4),
        'F1-Score':  round(f1_score(y_true, y_pred, average='weighted', zero_division=0), 4),
```

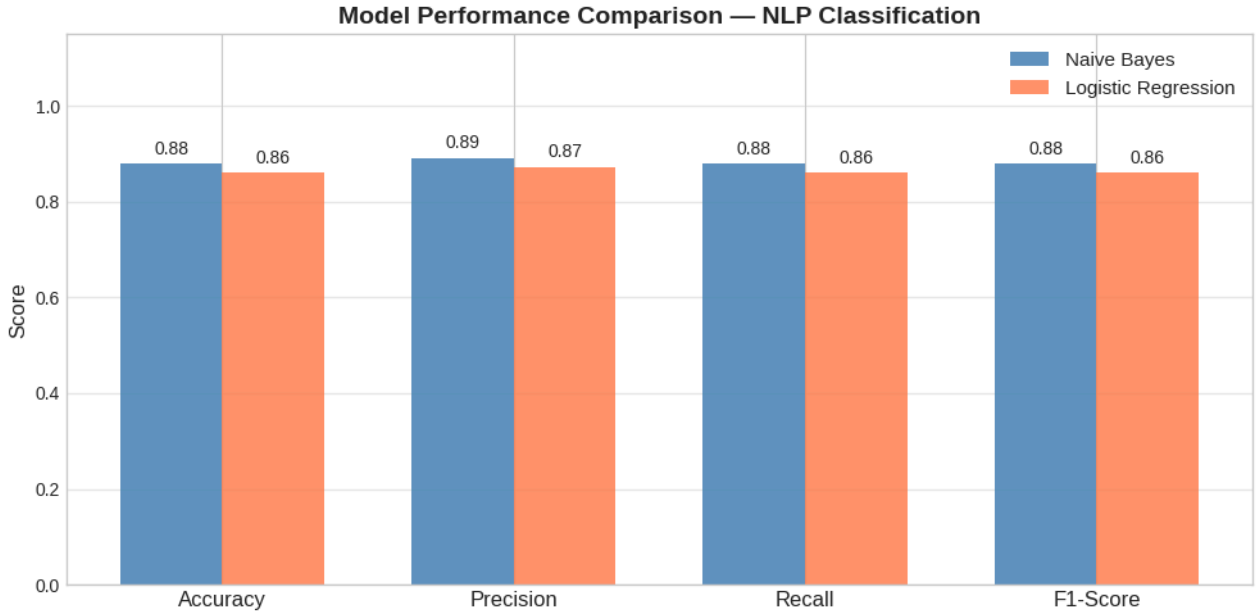
```
    }

results = pd.DataFrame([
    get_metrics(y_test, y_pred_nb, 'Naive Bayes (TF-IDF)'),
    get_metrics(y_test, y_pred_lr, 'Logistic Regression (TF-IDF)'),
])

print(" Model Performance Comparison Table:")
print("=" * 65)
print(results.to_string(index=False))
print("\n Higher values are better for all metrics.")

# Determine better model
nb_acc  = results.loc[0, 'Accuracy']
lr_acc  = results.loc[1, 'Accuracy']
winner  = 'Logistic Regression' if lr_acc >= nb_acc else 'Naive Bayes'
print(f"\n Better model: {winner} (Accuracy: {max(nb_acc, lr_acc):.2%})")
```

```
 Model Performance Comparison Table:
=================================================================
                      Model  Accuracy  Precision  Recall  F1-Score
        Naive Bayes (TF-IDF)      0.88     0.8920    0.88    0.8794
Logistic Regression (TF-IDF)      0.86     0.8729    0.86    0.8610

 Higher values are better for all metrics.

 Better model: Naive Bayes (Accuracy: 88.00%)
```

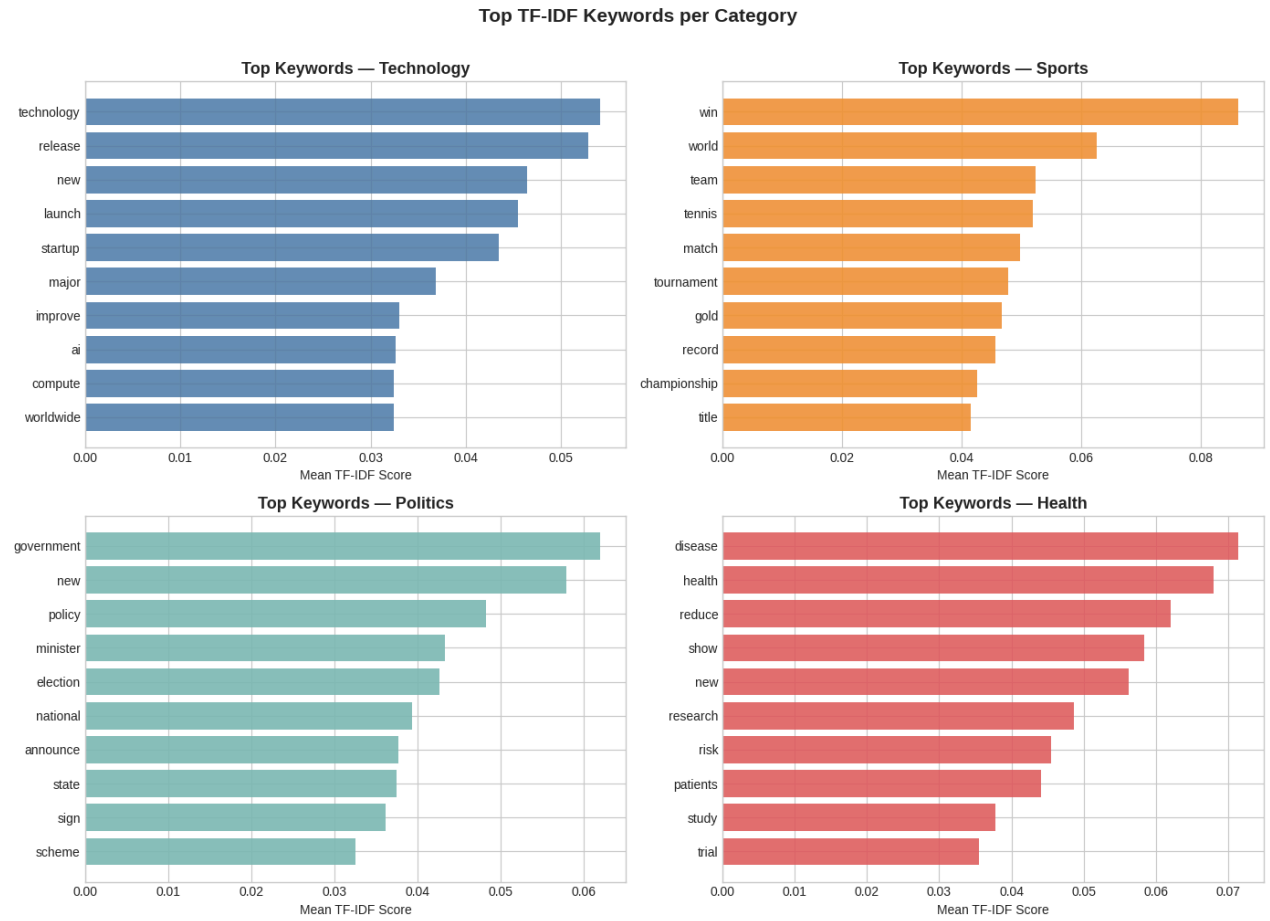
---

7. Visualizations

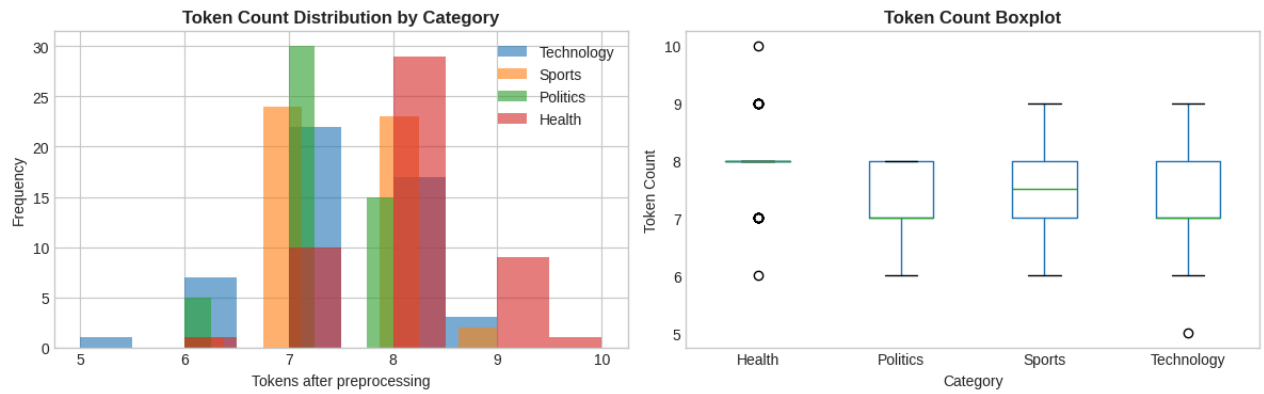
```
# ----------------------------------------------------------------------
# Plot 1: Metrics bar chart comparison
# ----------------------------------------------------------------------
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 5))
bars1 = ax.bar(x - width/2, results.loc[0, metrics], width, label='Naive Bayes', color='steelblue', alpha=0.85)
bars2 = ax.bar(x + width/2, results.loc[1, metrics], width, label='Logistic Regression', color='coral', alpha=0.85)

ax.set_ylim(0, 1.15)
ax.set_xticks(x)
ax.set_xticklabels(metrics, fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Model Performance Comparison — NLP Classification', fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
ax.bar_label(bars1, fmt='%.2f', padding=3, fontsize=10)
ax.bar_label(bars2, fmt='%.2f', padding=3, fontsize=10)
ax.grid(axis='y', alpha=0.4)

plt.tight_layout()
plt.savefig('metrics_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ----------------------------------------------------------------------
# Plot 2: Top TF-IDF keywords per category
# ----------------------------------------------------------------------
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()
colors = ['#4e79a7', '#f28e2b', '#76b7b2', '#e15759']

for idx, (cat_label, cat_name) in enumerate(label_map.items()):
    cat_indices = df[df['label'] == cat_label].index
    cat_matrix  = X_tfidf[cat_indices]
    mean_scores = cat_matrix.mean(axis=0).A1
    top_n = 10
    top_idx   = mean_scores.argsort()[::-1][:top_n]
    top_words = [tfidf_vec.get_feature_names_out()[i] for i in top_idx]
    top_vals  = [mean_scores[i] for i in top_idx]

    ax = axes[idx]
    ax.barh(top_words[::-1], top_vals[::-1], color=colors[idx], alpha=0.85)
    ax.set_title(f'Top Keywords — {cat_name}', fontsize=13, fontweight='bold')
    ax.set_xlabel('Mean TF-IDF Score', fontsize=10)
    ax.tick_params(axis='y', labelsize=10)

plt.suptitle('Top TF-IDF Keywords per Category', fontsize=15, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('tfidf_keywords.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ----------------------------------------------------------------------
# Plot 3: Token count distribution after preprocessing
# ----------------------------------------------------------------------
df['token_count'] = df['processed_text'].apply(lambda x: len(x.split()))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Distribution by category
for cat_label, cat_name in label_map.items():
    subset = df[df['label'] == cat_label]['token_count']
    axes[0].hist(subset, alpha=0.6, label=cat_name, bins=8)
axes[0].set_title('Token Count Distribution by Category', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Tokens after preprocessing')
axes[0].set_ylabel('Frequency')
axes[0].legend()

# Box plot
df.boxplot(column='token_count', by='category', ax=axes[1], grid=False)
axes[1].set_title('Token Count Boxplot', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Category')
axes[1].set_ylabel('Token Count')
plt.suptitle('')

plt.tight_layout()
plt.savefig('token_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
print(df.groupby('category')['token_count'].describe().round(2))
```




```
            count  mean   std  min  25%  50%  75%   max
category
Health       50.0  7.98  0.74  6.0  8.0  8.0  8.0  10.0
Politics     50.0  7.20  0.61  6.0  7.0  7.0  8.0   8.0
Sports       50.0  7.52  0.61  6.0  7.0  7.5  8.0   9.0
Technology   50.0  7.28  0.86  5.0  7.0  7.0  8.0   9.0
```

8. Prediction on New Samples

Test both models on **unseen** headlines — not in the training set.

```
new_headlines = [
    "NASA deploys new satellite using advanced rocket technology",       # Technology
    "India defeats Pakistan in high-scoring cricket ODI match",          # Sports
    "Government announces new tax reform policy for middle class",       # Politics
    "Scientists develop new drug to treat antibiotic-resistant bacteria", # Health
    "ChatGPT integrated into education platforms for student learning",  # Technology
    "Olympic torch relay begins across host city streets",               # Sports
]

# Preprocess new inputs using the same pipeline
new_processed = [preprocess_text(h) for h in new_headlines]
new_vec       = tfidf_vec.transform(new_processed)

pred_nb = nb_model.predict(new_vec)
pred_lr = lr_model.predict(new_vec)
proba_lr = lr_model.predict_proba(new_vec)

print(" Predictions on New Headlines:")
print("=" * 78)
print(f"  {'Headline':<55} {'NB':<12} {'LR (conf)':<15}")
print("  " + "-"*78)

for i, headline in enumerate(new_headlines):
    nb_cat   = label_map[pred_nb[i]]
    lr_cat   = label_map[pred_lr[i]]
    lr_conf  = proba_lr[i].max()
    match    = "✅" if pred_nb[i] == pred_lr[i] else "⚠️"
    print(f"  {headline[:52]:<55} {nb_cat:<12} {lr_cat} ({lr_conf:.0%}) {match}")
```

```
 Predictions on New Headlines:
==============================================================================
  Headline                                                NB           LR (conf)
  ------------------------------------------------------------------------------
  NASA deploys new satellite using advanced rocket tec    Technology   Technology (50%) ✅
  India defeats Pakistan in high-scoring cricket ODI m    Sports       Sports (51%) ✅
  Government announces new tax reform policy for middl    Politics     Politics (54%) ✅
  Scientists develop new drug to treat antibiotic-resi    Technology   Technology (32%) ✅
  ChatGPT integrated into education platforms for stud    Politics     Politics (31%) ✅
  Olympic torch relay begins across host city streets     Sports       Sports (47%) ✅
```

9. Summary & Conclusions

Preprocessing Steps Applied